

CPE 121 Mobile Application Developments

บทที่ 3

คำสั่งควบคุมทิศทางการทำงานของโปรแกรม (Control Statement)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.

Department of Computer Engineering

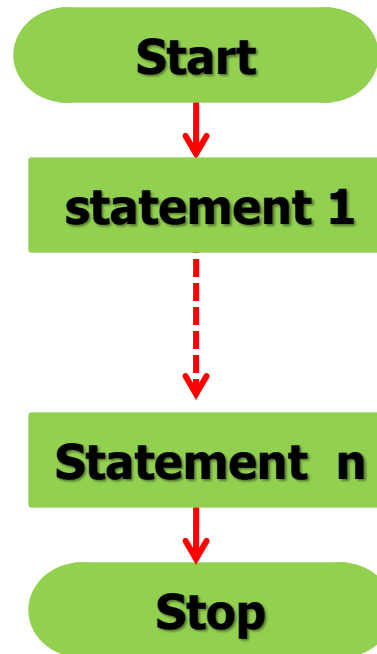
การกำหนดทิศทางการทำงานของโปรแกรม

- ใช้คำสั่งควบคุมซึ่งมี 3 รูปแบบ คือ
 - คำสั่งแบบตามลำดับ (Sequence Control Statement)
 - คำสั่งแบบมีทางเลือก (Selection Control Statement)
 - คำสั่ง `if` / `if_else` / `nested if` / `switch`
 - คำสั่งแบบทำซ้ำ (Iteration Control Statement)
 - คำสั่ง `while` / `do_while` / `for`
- คำสั่ง `break`
- คำสั่ง `continue`



คำสั่งแบบตามลำดับ (Sequence Control Statement)

- ทุกคำสั่งจะทำงานตามลำดับ โดยไม่มีการข้ามคำสั่งใด
- แต่ละคำสั่งจะถูกเรียกใช้งานเพียงครั้งเดียว
- การทำงานจะเริ่มจากคำสั่งที่ 1 แล้วทำคำสั่งที่ 2 และถัดไปเรื่อยๆ จนครบทุกคำสั่ง



คำสั่งแบบตามลำดับ (Sequence Control Statement)

```
import java.util.Scanner;

public class Bonus {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        System.out.print("ป้อนข้อมูลเงินเดือนที่นี่ >>> ");
        float salary = scan.nextFloat();
        float bonus = 0.05f*salary;
        System.out.println("โบนัส เท่ากับ " + bonus + " บาท");
    }
}
```

> run Bonus

ป้อนข้อมูลเงินเดือนที่นี่ >>> 20000

โบนัส เท่ากับ 1000.0 บาท



โปรแกรมการใช้งานคลาส Bonus_Calculator

- สร้างโปรเจกต์ **Bonus1**
- สร้าง Java Class ชื่อ **Bonus_Calculator** ไว้ในโปรเจกต์ **Bonus**
- สร้างเมธอดคำนวณเงินโบนัส (calBonus) รับค่าข้อมูลเงินเดือน ชนิด float และคืนค่าเงินโบนัสที่คำนวณได้ชนิด float
- สร้าง Java Main Class ชื่อ **Bonus1** ไว้ในโปรเจกต์ **Bonus1** เพื่อเรียกใช้คลาส **Bonus_Calculator**



โปรแกรมการใช้งานคลาส Bonus_Calculator

```
import java.util.Scanner;

public class Bonus1 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("ป้อนข้อมูลเงินเดือนที่นี่ >>> ");
        float salary = scan.nextFloat();
        Bonus_Calculator bonus = new Bonus_Calculator();
        float result = bonus.calBonus(salary);
        System.out.println("โบนัส เท่ากับ " + result + " บาท");
    }
}
```

```
public class Bonus_Calculator {
    public float calBonus(float s){
        return 0.05f*s;
    }
}
```

> run Bonus1

ป้อนข้อมูลเงินเดือนที่นี่ >>> 20000
โบนัส เท่ากับ 1000.0 บาท



คำสั่งแบบมีทางเลือก (Selection Control Statement)

- มีการตรวจสอบเงื่อนไขของคำสั่งก่อนทำงาน เพื่อตัดสินใจเลือกทิศทางการทำงานของโปรแกรม
- จะข้ามการทำงานของบางชุดคำสั่งไป มี 4 คำสั่งคือ
 - คำสั่ง `if`
 - คำสั่ง `if...else`
 - คำสั่ง `nested if`
 - คำสั่ง `switch`



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง if

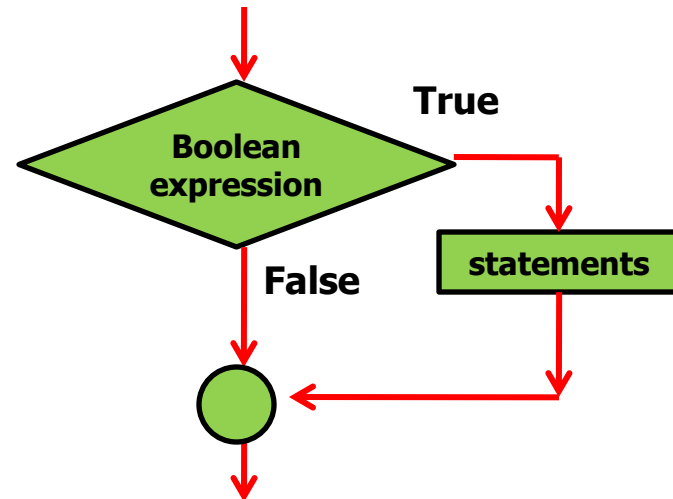
- ควบคุมให้โปรแกรมตัดสินใจทำงานหรือไม่ทำงานในชุดคำสั่งที่กำหนด
- ตรวจสอบจากนิพจน์ที่กำหนดว่าเป็นจริงหรือเท็จ
 - ถ้านิพจน์ เป็นจริง (**true**) โปรแกรมจะทำงานที่ชุดคำสั่งที่อยู่ภายใต้คำสั่ง if
 - ถ้านิพจน์ เป็นเท็จ (**false**) โปรแกรมจะข้ามไปทำงานที่คำสั่งต่อไปทันที
- มีรูปแบบการทำงานดังนี้

```
if (boolean_expression) {
    statements;
}
```

โดยที่

boolean expression
statements

เป็นนิพจน์เงื่อนไข ซึ่งมีผลการตรวจสอบเป็น **true** หรือ **false**
เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขที่กำหนดให้เป็น **true**



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง **if** : โปรแกรมคำนวณคะแนนร้อยละ เมื่อกดเลข 1 บนแป้นพิมพ์

```
import java.util.Scanner;

public class If_Test {

    public static void main(String[] args) {

        int key; float score, percent;

        Scanner scan = new Scanner(System.in);

        System.out.print("Press 1 to continue >>> ");

        key = scan.nextInt();

        if (key == 1) {

            System.out.print("Enter Score >>> ");

            score = scan.nextFloat();

            percent = score / 30.0f * 100;

            System.out.println("Your Score = " + percent + "%");

        }

        System.out.println("Good Bye !!! ");

    }

}
```

> run If_Test

Press 1 to continue >>> 1

Enter Score >>> 5000

Your Score = 16666.668%

Good Bye !!!



โปรแกรมการใช้งานคลาส ifTest_Condition

- สร้างโปรเจกต์ **If_Test1**
- สร้าง Java Class ชื่อ **ifTest_Condition** ไว้ในโปรเจกต์ **If_Test1**
- สร้างเมธอดตรวจสอบค่าตัวเลขชื่อ **checkValue** รับค่าข้อมูลตัวเลขเป็นชนิด **int** และคืนค่าผลการตรวจสอบเป็นชนิด **boolean**
- สร้าง Java Main Class ชื่อ **If_Test1** ไว้ในโปรเจกต์ **If_Test1** เพื่อเรียกใช้คลาส **ifTest_Condition**



โปรแกรมการใช้งานคลาส ifTest_Condition

```
import java.util.Scanner;

public class If_Test1 {

    public static void main(String[] args) {
        int key; float score, percent;

        Scanner scan = new Scanner(System.in);
        System.out.print("Press 1 to continue >>> ");
        key = scan.nextInt();

        ifTest_Condition iftest = new ifTest_Condition();
        boolean result = iftest.checkValue(key);

        if (result) {
            System.out.print("Enter Score >>> ");
            score = scan.nextFloat();
            percent = score / 30.0f * 100;
            System.out.println("Your Score = " + percent + "%");
        }
        System.out.println("Good Bye !!!");
    }
}
```

```
public class ifTest_Condition {

    public boolean checkValue(int value) {
        boolean check = false;

        if (value == 1)
            check = true;

        return check;
    }
}
```

> run If_Test1

Press 1 to continue >>> 1

Enter Score >>> 2000

Your Score = 6666.6665%

Good Bye !!!

คำสั่งแบบมีทางเลือก (Selection Control Statement)

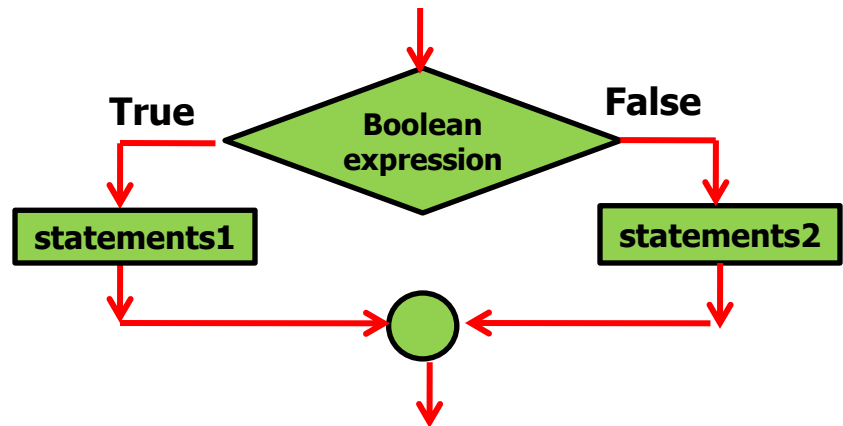
คำสั่ง if...else

- ควบคุมให้โปรแกรมเลือกทำงานในชุดคำสั่งใดชุดคำสั่งหนึ่งจาก 2 ทางเลือก
- ตรวจสอบจากนิพจน์ที่กำหนดว่าเป็นจริงหรือเท็จ
 - ถ้านิพจน์ เป็นจริง (**true**) โปรแกรมจะทำงานที่ชุดคำสั่งที่อยู่ภายใต้คำสั่ง **if**
 - ถ้านิพจน์ เป็นเท็จ (**false**) โปรแกรมจะทำงานที่ชุดคำสั่งที่อยู่ภายใต้คำสั่ง **else**
- มีรูปแบบการทำงานดังนี้

```
if (boolean expression) {
    statements 1;}
else{
    statements 2;}
```

โดยที่

boolean expression เป็นนิพจน์เงื่อนไข ซึ่งมีผลการตรวจสอบเป็น **true** หรือ **false**
statements 1 เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขที่กำหนดให้เป็น **true**
statements 2 เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขที่กำหนดให้เป็น **false**



คำสั่งแบบมีทางเลือก (Selection Control Statement)

▪ ข้อควรระวังในการใช้คำสั่ง if_else

- ในกรณีที่มีคำสั่งมากกว่า 1 คำสั่ง และไม่ใส่เครื่องหมาย { } ครบคำสั่งทั้งหมด จะทำให้โปรแกรมทำงานเฉพาะคำสั่งแรกเท่านั้น ทำให้ผลการทำงานผิดพลาดได้

```
if(a < b)
    System.out.println("a less than b");
else
    temp = a;
    a = b;    b = a;
    System.out.println("a and b are swap");
```

```
if(a > b)
    System.out.println("a less than b");
else{
    temp = a; }
    a = b;    b = a;
    System.out.println("a and b are swap");
```

- จากตัวอย่างโปรแกรมข้างต้น จะได้ผลการทำงานของโปรแกรมเหมือนกัน แต่จะแตกต่างจากโค้ดตัวอย่างโปรแกรมข้างล่างนี้

```
if(a < b)
    System.out.println("a less than b");
else {
    temp = a;
    a = b;    b = a;
    System.out.println("a and b are swap");}
```

- ควรใช้งานคำสั่ง if โดยกำหนดคำสั่งภายใต้เครื่องหมาย { } เสมอ



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง **if...else** : โปรแกรมตรวจสอบอายุเกษียณราชการของข้าราชการ

```
import java.util.Scanner;

public class If_else_Test {

    public static void main(String[] args) {

        int age;

        Scanner scan = new Scanner(System.in);

        System.out.print("Enter age >>> ");

        age = scan.nextInt();

        if (age > 60)

            System.out.println("Retired");

        else

            System.out.println("Not Retired");

    }

}
```

> run If_else_Test

Enter age >>> 60

Not Retired

> run If_else_Test

Enter age >>> 79

Retired



โปรแกรมการใช้งานคลาส ifelseTest_Condition

- สร้างโปรเจกต์ **If_else_Test1**
- สร้าง Java Class ชื่อ **ifelseTest_Condition** ไว้ในโปรเจกต์ **If_else_Test1**
- สร้างเมธอดตรวจสอบอายุ (checkRetired) รับค่าข้อมูลอายุเป็นชนิด int และคืนค่าผลการตรวจสอบเป็นข้อมูลชนิด boolean
- สร้าง Java Main Class ชื่อ **If_else_Test1** ไว้ในโปรเจกต์ **If_else_Test1** เพื่อเรียกใช้คลาส **ifelseTest_Condition**



โปรแกรมการใช้งานคลาส ifelseTest_Condition

```
import java.util.Scanner;

public class If_else_Test1 {

    public static void main(String[] args) {

        int age;

        Scanner scan = new Scanner(System.in);

        System.out.print("Enter age >>> ");

        age = scan.nextInt();

        ifelseTest_Condition ifelsetest = new ifelseTest_Condition();

        boolean result = ifelsetest.checkRetired(age);

        if (result)

            System.out.println("Retired");

        else

            System.out.println("Not Retired");

    }

}
```

```
public class ifelseTest_Condition {

    public boolean checkRetired(int age) {

        boolean check = false;

        if (age > 60)

            check = true;

        return check;

    }

}
```

> run If_else_Test1

Enter age >>> 52

Not Retired

> run If_else_Test1

Enter age >>> 64

Retired

คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง nested if หรือ if...else if

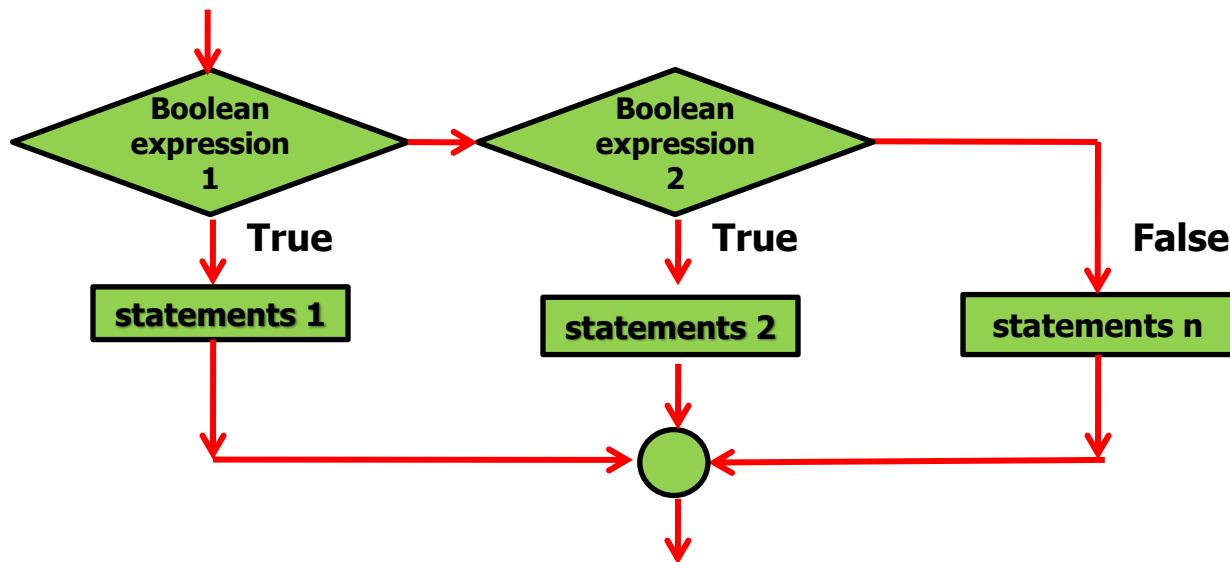
- ควบคุมให้โปรแกรมเลือกทำงานในชุดคำสั่งใดชุดคำสั่งหนึ่งจากหลายทางเลือก
- แต่ละทางเลือกจะมีการกำหนดนิพจน์เงื่อนไขเพื่อให้โปรแกรมตรวจสอบนิพจน์
 - หากพบว่าทางเลือกไหนมีนิพจน์เป็นจริง (true) ก็จะทำงานที่ชุดคำสั่งภายในทางเลือกนั้น โดยไม่พิจารณาทางเลือกอื่นที่ยังไม่ได้ทำการตรวจสอบอีก
- มีรูปแบบการทำงานดังนี้

```
if (boolean expression 1) {  
    statements 1;}  
else if (boolean expression 2) {  
    statements 2;}  
else {  
    statements n;}
```



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง nested if หรือ if...else if



โดยที่

boolean expression 1, **boolean expression 2** เป็นนิพจน์เงื่อนไขทางตรรกศาสตร์
statements 1 เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขของ boolean expression 1 เป็น **true**
statements 2 เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขของ boolean expression 2 เป็น **true**
statements n เป็นชุดของคำสั่งที่จะทำงานเมื่อเงื่อนไขที่กำหนดให้ทั้งหมดเป็น **false**



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง **nested if** หรือ **if...else if** : โปรแกรมตรวจสอบการเรียนครบตามหลักสูตรจากจำนวนหน่วยกิตวิชาเอกบังคับและวิชาเอกเลือก

```
import java.util.Scanner;

public class Nested_If_Test {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        System.out.print("จำนวนหน่วยกิตวิชาเอกบังคับ >>> ");

        int x = scan.nextInt();

        System.out.print("จำนวนหน่วยกิตวิชาเอกเลือก >>> ");

        int y = scan.nextInt();

        if (x >= 20 && y >= 30)

            System.out.println("ครบตามหลักสูตร");

        else if (x < 20 && y < 30)

            System.out.println("ขาดวิชาเอกบังคับ " + (20-x) + " หน่วยกิต\n" + "และวิชาเอกเลือก " + (30-y) + " หน่วยกิต");

        else if (x < 20)

            System.out.println("ขาดวิชาเอกบังคับ " + (20-x) + " หน่วยกิต");

        else if (y < 30)

            System.out.println("ขาดวิชาเอกเลือก " + (30-y) + " หน่วยกิต");

    }
}
```

> run Nested_If_Test

จำนวนหน่วยกิตวิชาเอกบังคับ >>> 18

จำนวนหน่วยกิตวิชาเอกเลือก >>> 25

ขาดวิชาเอกบังคับ 2 หน่วยกิต

และวิชาเอกเลือก 5 หน่วยกิต

โปรแกรมการใช้งานคลาส ifelseTest_Condition

- สร้างโปรเจกต์ **Nested_If_Test1**
- สร้าง Java Class ชื่อ **nestedifTest_Condition** ไว้ในโปรเจกต์ **Nested_If_Test1**
- สร้างเมธอดตรวจสอบจบ (checkPass) ซึ่งรับค่าข้อมูลจำนวนหน่วยกิตวิชาเอกบังคับและวิชาเอกเลือกเป็นชนิด int และไม่คืนค่ากลับให้แสดงผลในเมธอดนี้เลย
- สร้าง Java Main Class ชื่อ **Nested_If_Test1** ไว้ในโปรเจกต์ **Nested_If_Test1** เพื่อเรียกใช้คลาส **nestedifTest_Condition**



โปรแกรมการใช้งานคลาส ifelseTest_Condition

```
public class nestedifTest_Condition {
    public void checekPass(int a, int b) {
        if (a >= 20 && b >= 30)
            System.out.println("ครบตามหลักสูตร");
        else if (a < 20 && b < 30)
            System.out.println("ขาดวิชาเอกบังคับ " + (20-a) +
                " หน่วยกิต \n" + "และวิชาเอกเลือก " + (30-b) +
                " หน่วยกิต");
        else if (a < 20)
            System.out.println("ขาดวิชาเอกบังคับ " + (20-a) +
                " หน่วยกิต");
        else if (b < 30)
            System.out.println("ขาดวิชาเอกเลือก " + (30-b) +
                " หน่วยกิต");
    }
}
```

```
import java.util.Scanner;

public class Nested_If_Test1 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("จำนวนหน่วยกิตวิชาเอกบังคับ >>> ");
        int x = scan.nextInt();
        System.out.print("จำนวนหน่วยกิตวิชาเอกเลือก >>> ");
        int y = scan.nextInt();
        nestedifTest_Condition nestedif =
            new nestedifTest_Condition();
        nestedif.checekPass(x, y);
    }
}
```

> run Nested_If_Test1

จำนวนหน่วยกิตวิชาเอกบังคับ >>> 17

จำนวนหน่วยกิตวิชาเอกเลือก >>> 25

ขาดวิชาเอกบังคับ 3 หน่วยกิต

และวิชาเอกเลือก 5 หน่วยกิต

คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง switch statement

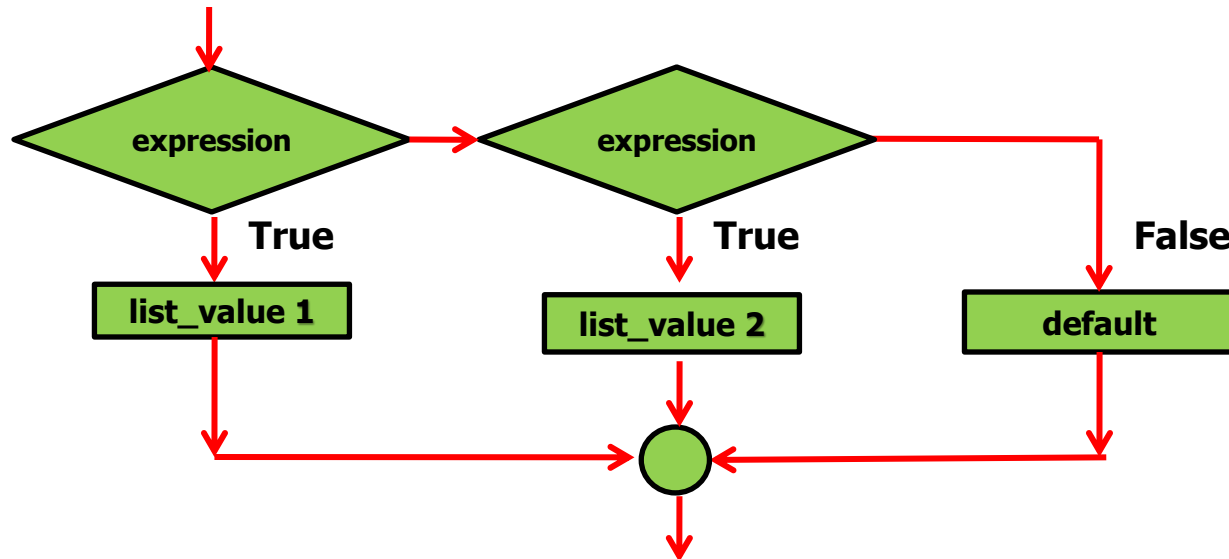
- ควบคุมให้โปรแกรมเลือกทำงานในชุดคำสั่งใดชุดคำสั่งหนึ่งจากหลายทางเลือก
- แต่ละทางเลือกจะมีการกำหนดเงื่อนไขของแต่ละทาง โดยตรวจสอบเงื่อนไขแต่ละทางเลือก
- หากพบว่าทางเลือกใดมีเงื่อนไขเป็นจริง (true) จะทำงานที่ชุดคำสั่งภายในทางเลือกนั้น โดยไม่พิจารณาทางเลือกอื่นที่ยังไม่ตรวจสอบอีก
- มีรูปแบบการทำงานดังนี้

```
switch (expression) {  
    case list_value_1:    statement_1; break;  
    case list_value_2:    statement_2; break;  
    case list_value_n:    statement_n; break;  
    default :            statement;  
}
```



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง switch statement



โดยที่

expression

เป็นนิพจน์ที่ต้องตรวจสอบค่าว่าตรงกับ **list_value** ใดเพื่อจะได้ทำงานตามชุดคำสั่งภายใต้ **list_value** นั้น

list_value

เป็นค่าข้อมูลที่ใช้ตรวจสอบกับ **expression**

statement_n

เป็นชุดของคำสั่งที่จะทำงานเมื่อ **list_value** นั้น มีค่าเท่ากับ **expression**

default

เป็นชุดของคำสั่งที่จะทำงานเมื่อไม่มี **list_value** ใด มีค่าเท่ากับ **expression**

คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง switch statement : โปรแกรมตรวจสอบการดำเนินการกับตัวเลข 2
ค่าให้ตรงกับตัวดำเนินการที่เลือก

```
import java.util.Scanner;

public class Switch_Test {

    public static void main(String args[]) {

        System.out.println("Main menu");

        System.out.println("1.Add");          System.out.println("2.Subtract");
        System.out.println("3.Multiply");      System.out.println("4.Divide");

        System.out.print("Press 1,2,3 or 4 >>> ");

        Scanner scan = new Scanner(System.in);  int key = scan.nextInt();

        System.out.print("enter first number >>> ");  float a = scan.nextFloat();

        System.out.print("enter seceond number >>> ");  float b = scan.nextFloat();
```



คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง switch statement : โปรแกรมตรวจสอบการดำเนินการกับตัวเลข 2 ค่าให้ตรงกับตัวดำเนินการที่เลือก

```
switch (key) {  
    case 1:    System.out.println("result of" + a + "+" + b + "=" + (a + b));  
              break;  
    case 2:    System.out.println("result of" + a + "-" + b + "=" + (a - b));  
              break;  
    case 3:    System.out.println("result of" + a + "*" + b + "=" + (a * b));  
              break;  
    case 4:    System.out.println("result of" + a + "/" + b + "=" + (a / b));  
              break;  
    default:   System.out.println("Unknown Operator !!!");  
}  
System.out.println("Good Bye !!! ");  
}
```

คำสั่งแบบมีทางเลือก (Selection Control Statement)

คำสั่ง **switch statement** : โปรแกรมตรวจสอบการดำเนินการกับตัวเลข 2
ค่าให้ตรงกับตัวดำเนินการที่เลือก

```
> run Switch_Test
```

```
Main menu
```

```
1.Add
```

```
2.Subtract
```

```
3.Multiply
```

```
4.Devide
```

```
Press 1,2,3 or 4 >>> 3
```

```
enter first number >>> 50
```

```
enter seceond number >>> 49
```

```
result of 50.0 * 49.0 = 2450.0
```

```
Good Bye !!!
```



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง while

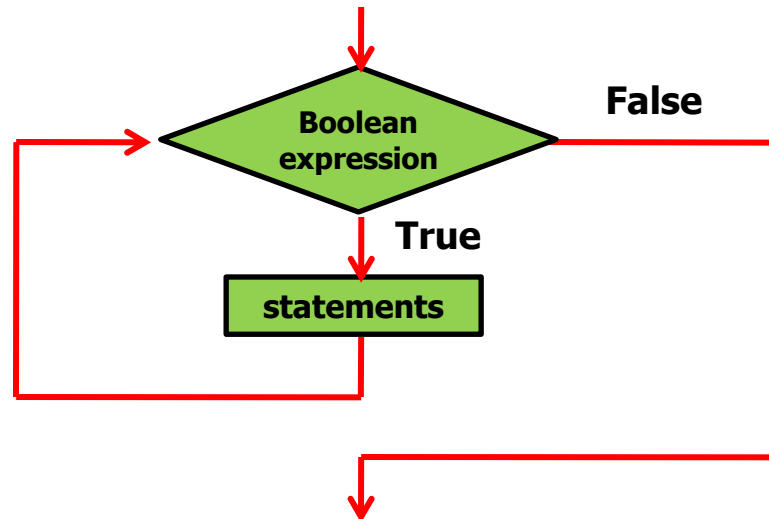
- มีการตรวจสอบเงื่อนไขที่เป็นนิพจน์ทางตรรกศาสตร์ก่อนการทำงานชุดคำสั่งภายในลูปทุกครั้ง
- โปรแกรมจะทำงานที่ชุดคำสั่งภายในลูปก็ต่อเมื่อนิพจน์ที่กำหนดเป็นจริง
- เมื่อโปรแกรมทำงานที่ชุดคำสั่งภายในลูปเสร็จสิ้น โปรแกรมจะตรวจสอบนิพจน์ใหม่อีกครั้ง จะหยุดการทำซ้ำเมื่อนิพจน์ที่กำหนดเป็นเท็จ
- มีรูปแบบการทำงานดังนี้

```
while (boolean_expression){  
    statements;  
}
```



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง while



โดยที่

boolean_expression

เป็นนิพจน์ที่เป็นเงื่อนไขทางตรรกศาสตร์มีการตรวจสอบก่อนการทำงานชุดคำสั่งภายในลูปทุกครั้ง

statements

เป็นชุดของคำสั่งที่ต้องการให้มีการทำซ้ำ



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง **while** : โปรแกรมแสดงผลการคำนวณผลรวมของ 1 ถึง 20

```
public class While_Test {  
    public static void main(String args[]) {  
        int count = 1, sum=0;  
        while (count <= 20) {  
            sum += count;  
            count++;  
        }  
        System.out.println("summation of 1 to 20 = " + sum);  
    }  
}
```

> run While_Test

summation of 1 to 20 = 210

คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง do...while

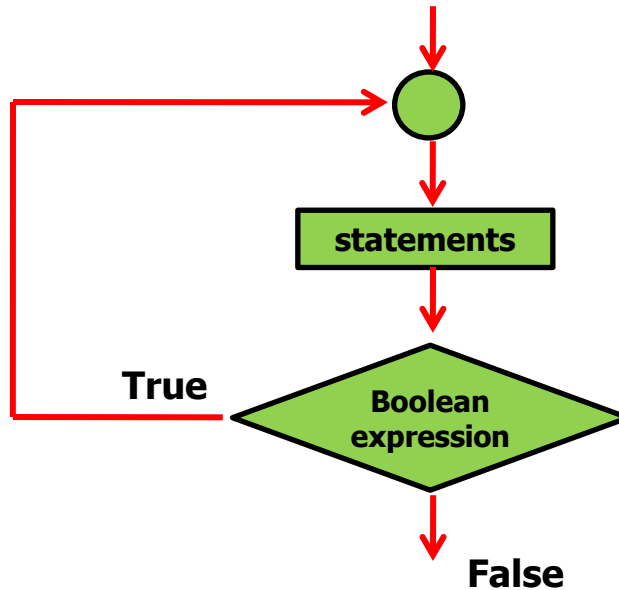
- มีการทำงานในชุดคำสั่งอย่างน้อย 1 รอบ ก่อนตรวจสอบเงื่อนไขที่เป็นนิพจน์ทางตรรกศาสตร์หลังคำสั่ง **while**
- ถ้านิพจน์ที่กำหนดเป็นจริง จะทำงานชุดคำสั่งภายในลูปอีกครั้ง ถ้านิพจน์ที่กำหนดเป็นเท็จโปรแกรมจะออกจากลูปการทำงานไปทำงานที่คำสั่งถัดไปทันที
- มีรูปแบบการทำงานดังนี้

```
do{  
    statements;  
} while (boolean_expression);
```



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง do...while



โดยที่

statements

เป็นชุดของคำสั่งที่ต้องการให้มีการทำซ้ำ

boolean_expression

ชุดคำสั่งเสร็จสิ้นทุกครั้ง

เป็นนิพจน์เงื่อนไขทางตรรกศาสตร์ที่มีการตรวจสอบหลังทำงานใน



Faculty of Engineering
SRINAKHARINWIROT UNIVERSITY

คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง do...while : โปรแกรมคำนวณผลรวมของเลขที่รับเข้ามาเป็นค่าทางแป้นพิมพ์

```
import java.util.Scanner;

public class Do_While_Test {
    public static void main(String[] args) {
        int num, sum=0;
        Scanner scan = new Scanner(System.in);
        do {
            System.out.print("Enter number >>> ");
            num = scan.nextInt();
            if (num >= 0)
                sum += num;
        } while (num >= 0);
        System.out.println("Result is " + sum);
    }
}
```

> run Do_While_Test

Enter number >>> 20

Enter number >>> 10

Enter number >>> -5

Result is 30



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง for

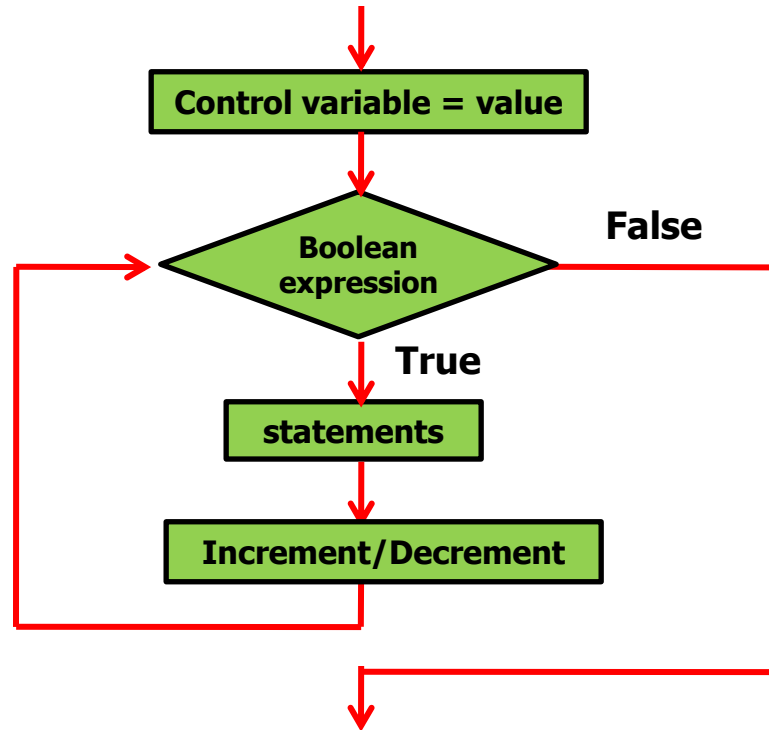
- มีการตรวจสอบเงื่อนไขที่เป็นนิพจน์ทางตรรกศาสตร์ก่อนการทำงานที่ชุดคำสั่งภายในลูปทุกครั้ง
- โปรแกรมจะทำงานที่ชุดคำสั่งภายในลูปเมื่อเงื่อนไขเป็นจริง ซึ่งจะมีการเพิ่มหรือลดค่าตัวแปร
- มีรูปแบบการทำงานดังนี้

```
for (control_variable = value; boolean_expression; increment or decrement){  
    statements;  
}
```



คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง for



โดยที่

control variable

value

boolean expression

increment or decrement

statements

เป็นตัวแปรที่ควบคุมจำนวนครั้งของการทำซ้ำ
มีชนิดเป็นจำนวนเต็ม

เป็นค่าเริ่มต้นที่กำหนดให้ตัวแปรควบคุม

เป็นนิพจน์เงื่อนไขทางตรรกศาสตร์ที่มีการตรวจสอบเงื่อนไข

เป็นคำสั่งกำหนดการเพิ่มค่าหรือลดค่าของตัวแปรควบคุม

เป็นชุดของคำสั่งที่ต้องการให้มีการทำซ้ำ

คำสั่งแบบทำซ้ำ (Iteration Control Statement)

คำสั่ง for : โปรแกรมคำนวณคะแนนเฉลี่ยของนักศึกษา จากข้อมูลคะแนนนักศึกษาจำนวน N คน

```
import java.util.Scanner;

public class For_Test_2 {

    public static void main(String args[]) {

        int N;    float avg = 0.0f, sum = 0.0f, score;

        Scanner scan = new Scanner(System.in);

        System.out.print("จำนวนนิสิต >>> ");

        N = scan.nextInt();

        for(int i=0; i<N; i++) {

            System.out.print("คะแนนสอบของนิสิตคนที่ " + (i+1) + " >>> ");

            score = scan.nextFloat(); sum=sum+score;

        }

        avg=sum/N;

        System.out.println("คะแนนเฉลี่ย = "+avg);

    }

}
```

> run For_Test_2

จำนวนนิสิต >>> 3

คะแนนสอบของนิสิตคนที่ 1 >>> 10

คะแนนสอบของนิสิตคนที่ 2 >>> 15

คะแนนสอบของนิสิตคนที่ 3 >>> 20

คะแนนเฉลี่ย = 15.0



คำสั่ง break

- เป็นคำสั่งที่ใช้ร่วมกับคำสั่งการทำซ้ำ เพื่อหยุดการทำงานก่อนครบตามจำนวนรอบที่กำหนด
- หรือใช้เป็นคำสั่งหยุดการทำงานภายในชุดคำสั่งของ **switch** เพื่อข้ามคำสั่งเงื่อนไขอื่นๆ ที่อยู่ภายในบล็อกของ **switch**
- ตัวอย่างโปรแกรมรับข้อมูลคะแนนของนักศึกษาจำนวน 5 คน ถ้าคะแนนน้อยกว่า 0 หรือ มากกว่า 100 ให้หยุดการรับข้อมูลคะแนน



คำสั่ง break

```
import java.util.Scanner;

public class Break_Test {

    public static void main(String args[]) {
        int i = 1, score = 100;
        Scanner scan = new Scanner(System.in);
        while (i <= 5) {
            System.out.print("ป้อนข้อมูลคะแนนนิสิตคนที่ " + i + " >>> ");
            score = scan.nextInt();
            if ((score > 100) || (score < 0)) {
                System.out.println(score + " เป็นข้อมูลคะแนนที่ไม่ถูกต้อง");
                break;
            }
            System.out.println(score + " เป็นข้อมูลคะแนนที่ถูกต้อง");
            i = i+1;
        }
    }
}
```

> run Break_Test

ป้อนข้อมูลคะแนนนิสิตคนที่ 1 >>> 95
 95 เป็นข้อมูลคะแนนที่ถูกต้อง
 ป้อนข้อมูลคะแนนนิสิตคนที่ 2 >>> 80
 80 เป็นข้อมูลคะแนนที่ถูกต้อง
 ป้อนข้อมูลคะแนนนิสิตคนที่ 3 >>> 90
 90 เป็นข้อมูลคะแนนที่ถูกต้อง
 ป้อนข้อมูลคะแนนนิสิตคนที่ 4 >>> 101
 101 เป็นข้อมูลคะแนนที่ไม่ถูกต้อง



คำสั่ง continue

- เป็นคำสั่งที่ทำงานตรงข้ามกับคำสั่ง **break**
- จะเป็นคำสั่งที่บังคับให้โปรแกรมข้ามไปทำงานในรอบต่อไปทันที โดยไม่สนใจคำสั่งที่เหลืออยู่ในลูป
- ตัวอย่างโปรแกรมรับข้อมูลคะแนนของนักศึกษาจำนวน 5 คน ถ้าคะแนนน้อยกว่า 0 หรือ มากกว่า 100 ให้รับข้อมูลคะแนนต่อไปจนกว่าจะครบ



คำสั่ง continue

```
import java.util.Scanner;

public class Continue_Test {

    public static void main(String args[]) {
        int i = 1, score = 100;
        Scanner scan = new Scanner(System.in);
        while (i <= 5) {
            System.out.print("ป้อนข้อมูลคะแนนนิสิตคนที่ " + i + " >>> ");
            score = scan.nextInt();
            if ((score > 100) || (score < 0)) {
                System.out.println(score + " เป็นข้อมูลคะแนนที่ไม่ถูกต้อง");
                continue;
            }
            System.out.println(score + " เป็นข้อมูลคะแนนที่ถูกต้อง");
            i = i+1;
        }
    }
}
```

> run Continue_Test

```
ป้อนข้อมูลคะแนนนิสิตคนที่ 1 >>> 90
90 เป็นข้อมูลคะแนนที่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 2 >>> 95
95 เป็นข้อมูลคะแนนที่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 3 >>> 80
80 เป็นข้อมูลคะแนนที่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 4 >>> 115
110 เป็นข้อมูลคะแนนที่ไม่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 4 >>> 93
93 เป็นข้อมูลคะแนนที่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 5 >>> 111
111 เป็นข้อมูลคะแนนที่ไม่ถูกต้อง
ป้อนข้อมูลคะแนนนิสิตคนที่ 5 >>> 97
97 เป็นข้อมูลคะแนนที่ถูกต้อง
```

คำสั่ง **break** และ **continue**

- คำสั่ง **break** และคำสั่ง **continue** เป็นคำสั่งที่ใช้ร่วมกับคำสั่งการทำซ้ำเหมือนกัน แต่มีข้อแตกต่างกันคือ
 - คำสั่ง **break** จะหยุดการทำงานและออกจากการทำงานในลูปทันที
 - คำสั่ง **continue** จะบังคับให้โปรแกรมข้ามไปทำงานที่รอบต่อไปทันที
 - เมื่อพิจารณาจากผลการทำงานของตัวอย่างเมื่อคะแนนของนิสิตคนที่ 4 ไม่ถูกต้องจะได้ว่า
 - คำสั่ง **break** จะหยุดการทำงานและออกจากคำสั่ง **while** ทันที ทำให้มีการทำซ้ำทั้งหมด 4 ครั้ง
 - คำสั่ง **continue** จะบังคับให้ข้ามไปทำงานที่รอบต่อไปทันที แต่ยังเป็นการรับข้อมูลของนิสิตคนที่ 4 ทำให้มีการทำซ้ำทั้งหมด 6 ครั้ง

